In [1]:
```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.cluster import KMeans
from sklearn.decomposition import PCA
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import silhouette_score, classification_report, confusion_matrix, ConfusionMatrixDisplay
```

In [2]:
```
data = pd.read_csv('iris.csv')
display(data.head())
```

	SepalLength	SepalWidth	PetalLength	PetalWidth	Name
0	5.1	3.5	1.4	0.2	Iris-setosa
1	4.9	3.0	1.4	0.2	Iris-setosa
2	4.7	3.2	1.3	0.2	Iris-setosa
3	4.6	3.1	1.5	0.2	Iris-setosa
4	5.0	3.6	1.4	0.2	Iris-setosa

In [3]:
```
# Basic Info and Statistics
print("Dataset Info:")
data.info()
print("\nDataset Description:")
data.describe()
```

```
Dataset Info:
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 150 entries, 0 to 149
Data columns (total 5 columns):
 #   Column       Non-Null Count  Dtype  
---  ------       --------------  -----  
 0   SepalLength  150 non-null    float64
 1   SepalWidth   150 non-null    float64
 2   PetalLength  150 non-null    float64
 3   PetalWidth   150 non-null    float64
 4   Name         150 non-null    object 
dtypes: float64(4), object(1)
memory usage: 6.0+ KB

Dataset Description:
```

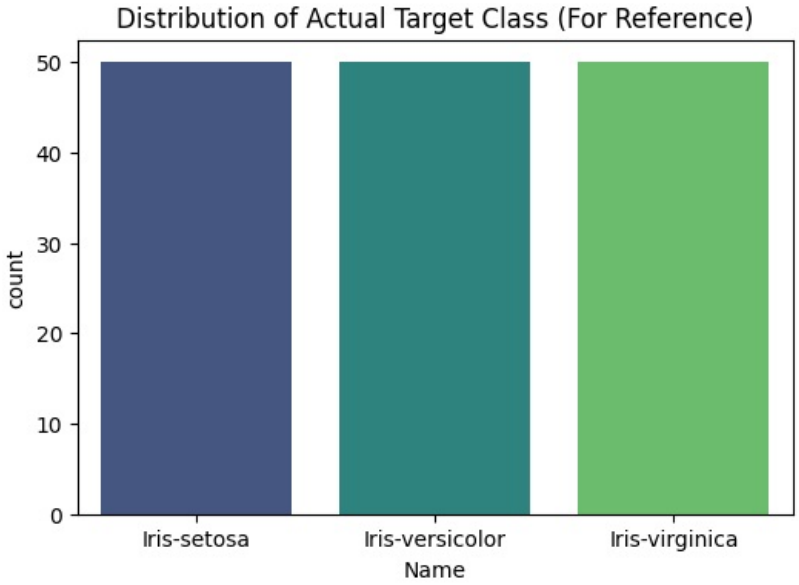
Out[3]:

	SepalLength	SepalWidth	PetalLength	PetalWidth
count	150.000000	150.000000	150.000000	150.000000
mean	5.843333	3.054000	3.758667	1.198667
std	0.828066	0.433594	1.764420	0.763161
min	4.300000	2.000000	1.000000	0.100000
25%	5.100000	2.800000	1.600000	0.300000
50%	5.800000	3.000000	4.350000	1.300000
75%	6.400000	3.300000	5.100000	1.800000
max	7.900000	4.400000	6.900000	2.500000

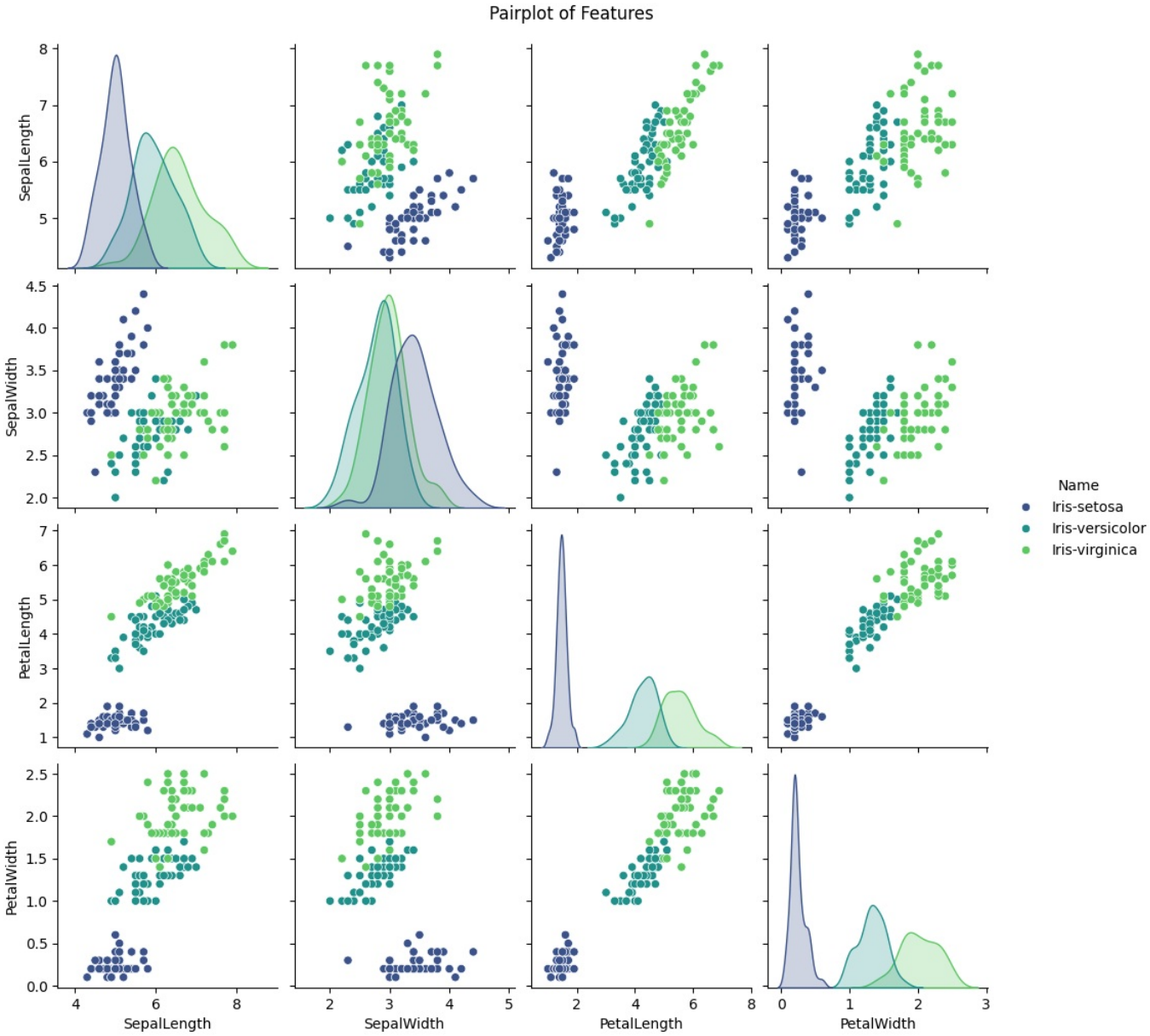
In [4]:
```
# Check for missing values
print("Missing Values:")
print(data.isnull().sum())
```

```
Missing Values:
SepalLength    0
SepalWidth     0
PetalLength    0
PetalWidth     0
Name           0
dtype: int64
```

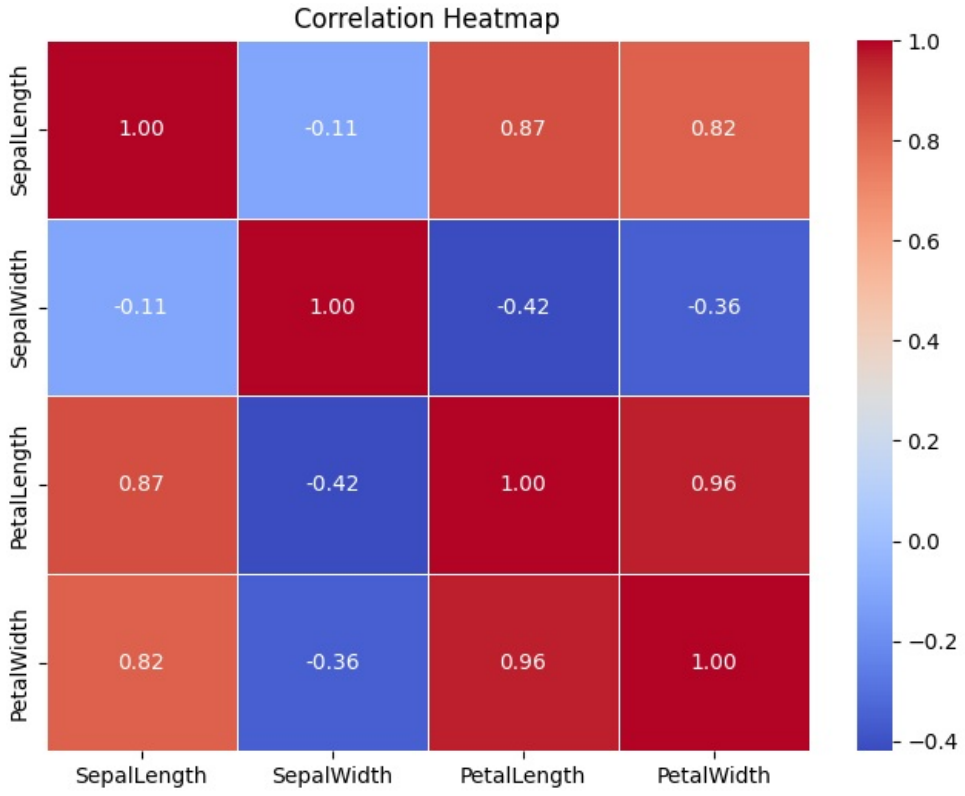
In [5]:
```
# Distribution of Target Class (Actual Labels for reference)
plt.figure(figsize=(6,4))
sns.countplot(x='Name', data=data, hue='Name', palette='viridis')
plt.title('Distribution of Actual Target Class (For Reference)')
plt.show()
```



In [6]:
```
# Pairplot for Feature Distributions and Relationships
sns.pairplot(data, hue='Name', palette='viridis', diag_kind='kde')
plt.suptitle('Pairplot of Features', y=1.02)
plt.show()
```



In [7]:
```
# Correlation Heatmap
numeric_cols = data.select_dtypes(include=[np.number]).columns
plt.figure(figsize=(8,6))
sns.heatmap(data[numeric_cols].corr(), annot=True, cmap='coolwarm', fmt=".2f", linewidths=0.5)
plt.title('Correlation Heatmap')
plt.show()
```

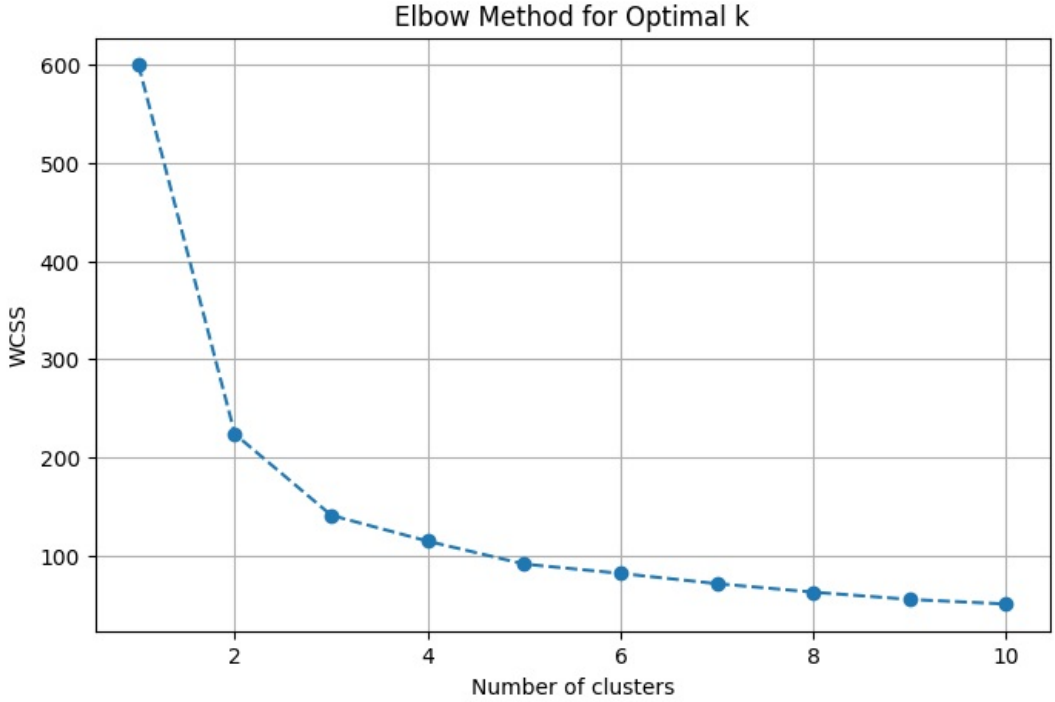


In [8]:
```
X = data.drop('Name', axis=1)
y = data['Name'] # We keep this to compare clusters with actual classes

# Standardizing the features for K-Means
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)
```

In [9]:
```
# Finding optimal number of clusters using Elbow Method
wcss = []
for i in range(1, 11):
    kmeans = KMeans(n_clusters=i, init='k-means++', max_iter=300, n_init=10, random_state=42)
    kmeans.fit(X_scaled)
    wcss.append(kmeans.inertia_)

plt.figure(figsize=(8, 5))
plt.plot(range(1, 11), wcss, marker='o', linestyle='--')
plt.title('Elbow Method for Optimal k')
plt.xlabel('Number of clusters')
plt.ylabel('WCSS')
plt.grid(True)
plt.show()
```



In [10]:
```
# Initialize and apply K-Means Clustering
k = 3 # Based on elbow method and dataset knowledge (Iris has 3 classes)
kmeans = KMeans(n_clusters=k, init='k-means++', max_iter=300, n_init=10, random_state=42)
cluster_labels = kmeans.fit_predict(X_scaled)

data['Cluster'] = cluster_labels
```

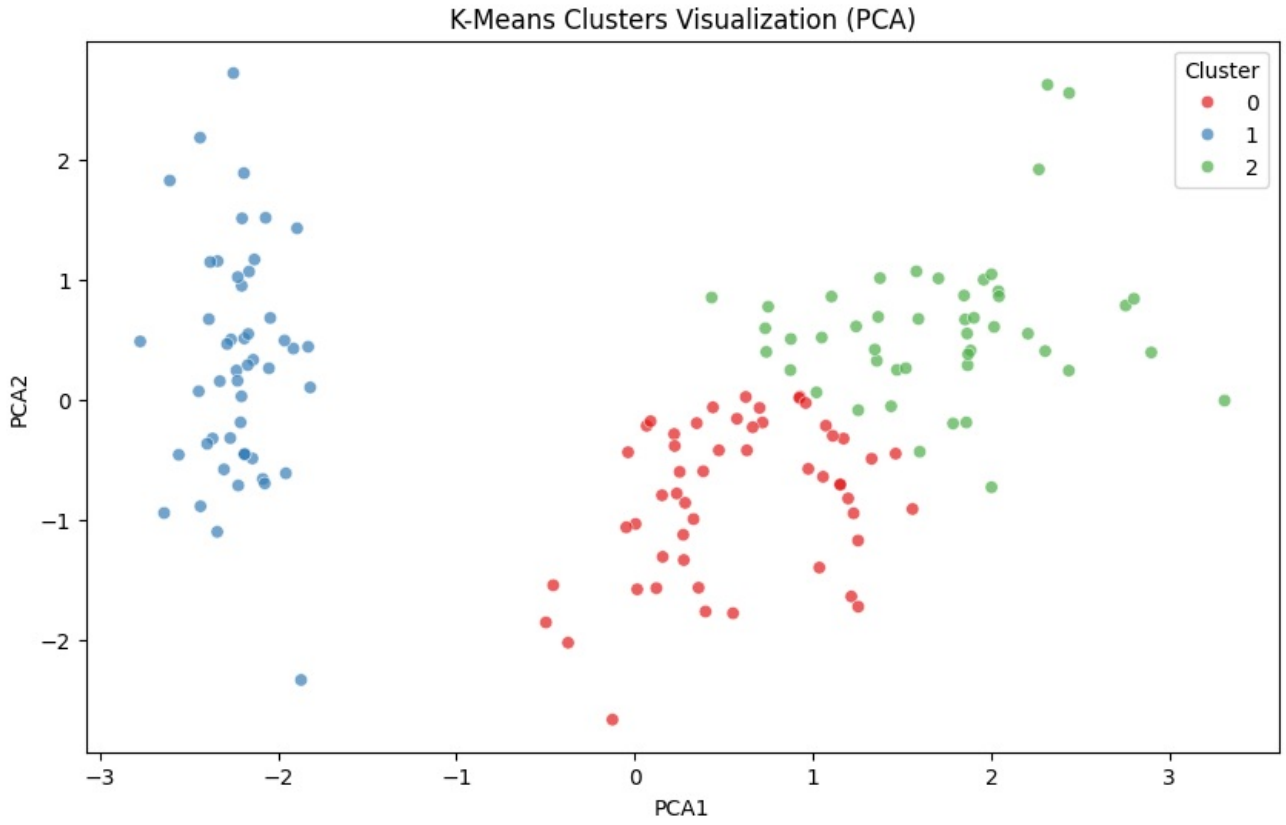
In [11]:
```
# Silhouette Score Evaluation
sil_score = silhouette_score(X_scaled, cluster_labels)
print(f"Silhouette Score: {sil_score:.4f}")
```

```
Silhouette Score: 0.4590
```

In [12]:
```
# Visualize Clusters using PCA (2D)
pca = PCA(n_components=2)
X_pca = pca.fit_transform(X_scaled)

data['PCA1'] = X_pca[:, 0]
data['PCA2'] = X_pca[:, 1]

plt.figure(figsize=(10, 6))
sns.scatterplot(x='PCA1', y='PCA2', hue='Cluster', data=data, palette='Set1', alpha=0.7)
plt.title('K-Means Clusters Visualization (PCA)')
plt.show()
```



In [13]:
```
# Comparing Clusters with Actual Labels
# Mapping clusters to actual classes (heuristic matching based on majority vote)
mapping = {}
for i in range(k):
    cluster_indices = np.where(cluster_labels == i)[0]
    actual_labels_in_cluster = y.iloc[cluster_indices]
    most_frequent_label = actual_labels_in_cluster.mode()[0]
    mapping[i] = most_frequent_label

predicted_labels = np.array([mapping[cluster] for cluster in cluster_labels])

print("--- Comparison of Clusters with Actual Classes ---")
print("Classification Report:")
print(classification_report(y, predicted_labels))
```

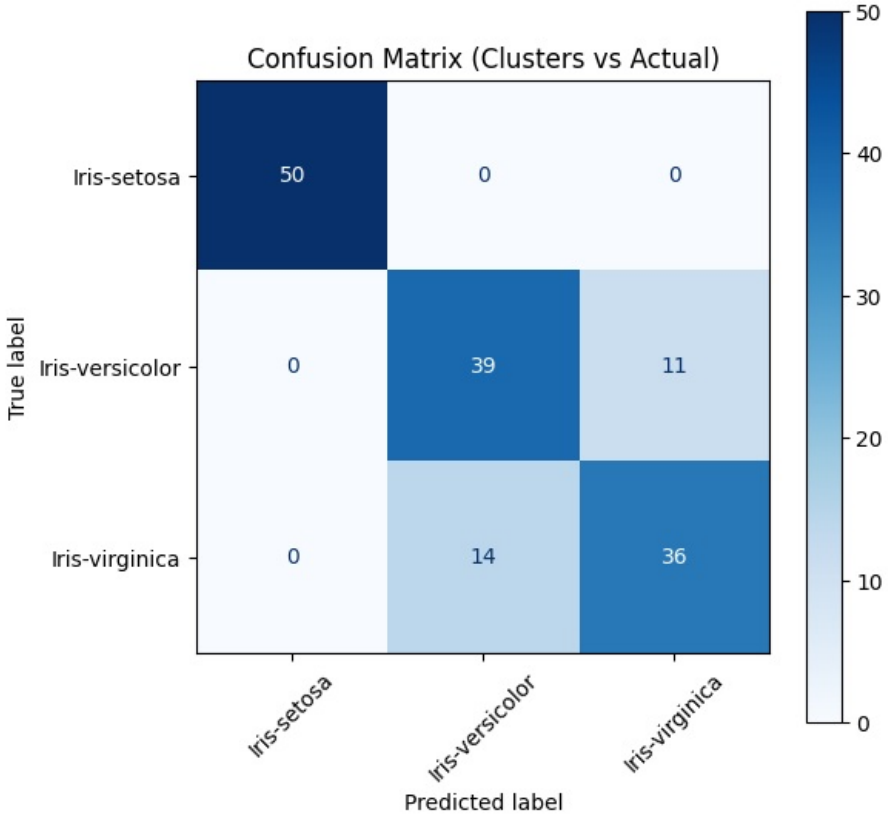
```
--- Comparison of Clusters with Actual Classes ---
Classification Report:
                 precision    recall  f1-score   support

    Iris-setosa       1.00      1.00      1.00        50
Iris-versicolor       0.74      0.78      0.76        50
 Iris-virginica       0.77      0.72      0.74        50

       accuracy                           0.83       150
      macro avg       0.83      0.83      0.83       150
   weighted avg       0.83      0.83      0.83       150
```

In [14]:
```python
# Confusion Matrix between Clusters and Actual Labels
cm = confusion_matrix(y, predicted_labels)
classes = np.unique(y)
disp = ConfusionMatrixDisplay(confusion_matrix=cm, display_labels=classes)

fig, ax = plt.subplots(figsize=(6, 6))
disp.plot(cmap='Blues', ax=ax, values_format='d')
plt.title('Confusion Matrix (Clusters vs Actual)')
plt.xticks(rotation=45)
plt.show()
```



In [15]:
```python
# Example assignment on a new data point
import pandas as pd

# Define the data
sample_data = [[5.1, 3.5, 1.4, 0.2]]
sample_df = pd.DataFrame(sample_data, columns=X.columns)

# Scale new data
sample_scaled = scaler.transform(sample_df)

# Predict Cluster
predicted_cluster = kmeans.predict(sample_scaled)[0]
predicted_mapped_class = mapping[predicted_cluster]

print(f"Sample Data: {sample_data[0]}")
print(f"Assigned Cluster: {predicted_cluster}")
print(f"Mapped Predicted Class: {predicted_mapped_class}")
```

```
Sample Data: [5.1, 3.5, 1.4, 0.2]
Assigned Cluster: 1
Mapped Predicted Class: Iris-setosa
```